

```
1
>>> True == 1
True
>>> False == 0
True
```

---

You can also use `isinstance()` to confirm that a Boolean value is considered a type of integer:

```
>>> isinstance(True, bool)
True
>>> isinstance(True, int)
True
```

---

The value `True` is of the `bool` data type. But because `bool` is a subclass of `int`, `True` is also an `int`. This means you can use `True` and `False` in almost any place you can use integers. This can lead to some bizarre code:

```
>>> True + False + True + True # Same as 1 + 0 + 1 + 1
3
>>> -True # Same as -1.
-1
>>> 42 * True # Same as 42 * 1 mathematical multiplication.
42
>>> 'hello' * False # Same as 'hello' * 0 string replication.
''
>>> 'hello'[False] # Same as 'hello'[0]
'h'
>>> 'hello'[True] # Same as 'hello'[1]
'e'
>>> 'hello'[-True] # Same as 'hello'[-1]
'o'
```

---

Of course, just because you can use `bool` values as numbers doesn't mean you should. The previous examples are all unreadable and should never be used in real-world code. Originally, Python didn't have a `bool` data type. It didn't add Booleans until Python 2.3, at which point it made `bool` a subclass of `int` to ease the implementation. You can read the history of the `bool` data type in PEP 285 at <https://www.python.org/dev/peps/pep-0285/>.

Incidentally, `True` and `False` were only made keywords in Python 3. This means that in Python 2, it was possible to use `True` and `False` as variable